

# AI Enhanced Edge IOT system for hands free control and energy monitoring

ATIKSH SINGH

*Computer Science and Engineering*  
*School of Computer Science*  
Phagwada, Punjab, India  
[atiksh.singhh19@gmail.com](mailto:atiksh.singhh19@gmail.com)

ARJUN SINGH

*Computer Science and Engineering*  
*School of Computer Science*  
Phagwada, Punjab, India  
[singharjun7002@gmail.com](mailto:singharjun7002@gmail.com)

MANISH NEHRA

*Computer Science and Engineering*  
*School of Computer Science*  
Phagwada, Punjab, India  
[nehramanish50@gmail.com](mailto:nehramanish50@gmail.com)

RISHABH JAIN

*Computer Science and Engineering*  
*School of Computer Science*  
Phagwada, Punjab, India  
[rishabhjain08040@gmail.com](mailto:rishabhjain08040@gmail.com)

## Abstract

This project introduces an intelligent home automation and safety system that enables contactless appliance control using hand gestures while continuously monitoring electrical energy usage. The system measures current, power, and total energy consumed using the ACS712 current sensor and displays the values on both an LCD display and the Blynk IoT application via Wi-Fi. To enhance safety, an MQ135 gas sensor is integrated to detect LPG or smoke. When gas concentration exceeds a defined threshold, the system triggers a buzzer alert and automatically switches on an exhaust fan to reduce gas accumulation, while the Blynk app updates the gas status in real time. The app also visualizes detailed graphs such as power vs time, energy vs time, current vs time, and gas level vs time, offering deeper analytical insight to the user. By combining gesture-based control, IoT-enabled monitoring, and automated safety response, this project contributes to a modern smart-home framework prioritizing convenience, efficiency, and user protection.

Keywords: AI Gesture Control, IoT, Energy Monitoring, Smart Home, MQ135, Home Automation

## Problem Statement:

- The increasing use of electrical appliances and electronic systems has raised the need for continuous monitoring of electrical current consumption to prevent energy wastage and electrical faults.
- Gas leakage and air pollution in indoor environments pose serious safety risks to human life, especially in homes, laboratories, and small industries, where early detection is often inadequate.
- Existing monitoring systems generally operate in isolation and lack an integrated solution that can simultaneously monitor gas levels, current consumption, and provide automated safety responses.
- Manual inspection and conventional safety methods are time-consuming, unreliable, and incapable of providing real-time alerts during emergency conditions such as toxic gas leakage or abnormal current flow.

- Many available commercial systems are expensive, bulky, and complex, making them unsuitable for low-cost and small-scale applications.
- There is a need for a compact, low-cost, and efficient embedded system that can monitor gas concentration and electrical current in real time while providing visual and audible alerts.
- The system should also be capable of automatically activating safety mechanisms, such as alarms and exhaust fans, to reduce hazards without human intervention.
- Hence, this project focuses on the design and implementation of an ESP32-based integrated monitoring and safety system that ensures improved environmental safety, energy awareness, and rapid response in domestic and small industrial environments.

## I. INTRODUCTION

### A. Background of Study and Motivation

Home automation has evolved from simple electrical switching to advanced IoT-based systems capable of remote control and monitoring. With increasing demand for convenience, safety, and energy efficiency, modern smart homes are expected to support intuitive appliance operation, real-time energy awareness, and automatic emergency response. Traditional systems, however, still rely heavily on manual switches and lack integrated safety mechanisms, making them less effective in addressing everyday household challenges.

One of the key motivations behind this project is the need for touchless and hygienic control of appliances, especially in environments like kitchens, hospitals, and for elderly or differently-abled individuals who may face difficulty reaching physical switches. Another major concern is rising electricity consumption, which requires users to understand their usage patterns to prevent energy wastage. Furthermore, LPG or smoke leakage incidents continue to pose serious risks in households, highlighting the need for proactive safety solutions rather than just reactive alerts.

This project aims to bridge these needs by developing a system that combines gesture-based appliance control, real-time energy

monitoring using the ACS712 current sensor, and automatic gas response using the MQ135 sensor. The system not only detects gas leakage and triggers a buzzer warning but also automatically switches on an exhaust fan to reduce gas concentration. Additionally, all live data—including gas level, current, power, and total energy consumed—is uploaded to the Blynk IoT application, allowing users to monitor their home remotely with real-time analytics and graph visualization.

The motivation behind this work is to create a complete smart-home framework that improves comfort, enhances safety, and promotes energy-conscious living through advanced sensor integration and IoT connectivity.

## B. Literature Review

Smart home automation has gained significant attention with the growth of IoT, embedded systems, and low-cost sensors. Several works propose gesture-based control of domestic loads to improve accessibility and hygiene, replacing conventional switches with hand movements captured by sensors or cameras [1]–[4]. Systems demonstrate that gesture interfaces are practical for controlling lights and fans, especially for elderly or differently-abled users, but they generally focus only on switching and do not incorporate energy analytics or gas-safety features.

In parallel, extensive research has explored energy monitoring using Hall-effect current sensors such as the ACS712, which provides galvanically isolated, linear analog output proportional to load current [5]–[7]. DIY and academic projects show ACS712 can be integrated with microcontrollers to measure AC/DC current, estimate instantaneous power, and compute total energy consumption for household loads [8]. However, most of these works present stand-alone meters or power-strip monitors rather than integrating current sensing with advanced control or safety mechanisms in a unified smart-home platform.

Gas and air-quality monitoring is another important research direction. The MQ135 gas sensor is widely used for detecting a range of harmful gases and monitoring air quality at low cost [9], [10]. Numerous LPG and smoke leakage detector designs employ MQ-series sensors with microcontrollers to trigger buzzers or LEDs when concentration exceeds a threshold, aiming to prevent domestic accidents [11]–[13]. These systems emphasize early warning but usually stop at audible/visual alerts and do not automatically activate exhaust fans or ventilation systems to mitigate the hazard.

With the emergence of IoT platforms, Blynk-based home automation has become popular for remote control and monitoring of appliances. Blynk provides cloud connectivity, mobile dashboards, and device management for ESP32/ESP8266-based projects [14]. Several works demonstrate controlling relays and monitoring environmental parameters via Blynk on ESP32, enabling smartphone-based operation of lights, fans, and other loads [15]–[17]. Recent research further explores ESP32-Blynk systems for more advanced smart-home scenarios, highlighting reliability of Wi-Fi connectivity and the usefulness of real-time mobile dashboards [18]. Still, most projects limit themselves either to remote switching or simple sensor visualization and rarely integrate gesture interfaces, detailed energy sensing, and gas-safety actions together.

From this review, it is evident that existing literature treats gesture-based control, energy monitoring with ACS712, gas detection using MQ135, and Blynk-based IoT dashboards largely as separate

problem domains. There is limited work that fuses these aspects into a single, low-cost smart-home system that (i) controls appliances via hand gestures, (ii) measures current, power, and total energy consumed, (iii) detects LPG/smoke levels with MQ135 and triggers both buzzer and automatic exhaust fan, and (iv) publishes real-time values and historical graphs (energy vs time, current vs time, power vs time, gas level vs time) to a Blynk mobile application. The present project addresses this gap by designing an integrated architecture that combines intuitive control, energy awareness, and active safety response within one ESP32-based platform.

## C. Research Objectives

The primary aim of this research is to design and implement an integrated smart-home system that enhances convenience, safety, and energy efficiency. The specific objectives of the study are:

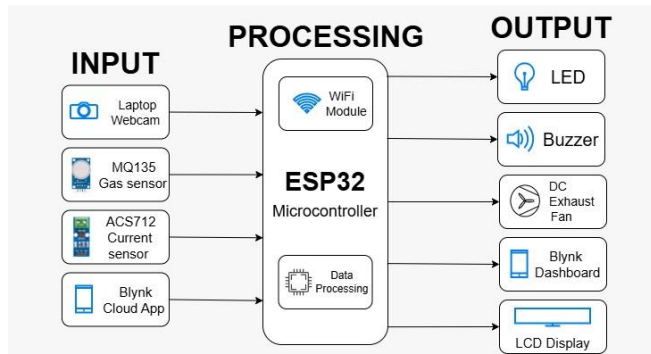
1. To develop a gesture-based appliance control mechanism that enables touch-free operation of household devices, providing a hygienic and user-friendly interface.
2. To monitor electrical parameters in real time, including current, power, and total energy consumption, using the ACS712 sensor for promoting energy-conscious behaviour.
3. To detect LPG/smoke concentration using the MQ135 sensor and trigger a buzzer alert when the gas level exceeds a predefined safety threshold.
4. To automatically activate an exhaust fan during high gas concentration, providing an immediate preventive response instead of relying only on alarm-based notifications.
5. To transmit all live sensor readings and appliance status to the Blynk IoT application via Wi-Fi, enabling remote visual monitoring and real-time graph analytics.
6. To integrate automation, energy monitoring, and safety in a single smart-home system, thereby improving overall functionality compared to existing standalone solutions

# II. METHODOLOGY AND MODELING

The system integrates:

1. **Gesture Control Module**  
Recognizes gestures using AI to turn appliances ON/OFF.  
Microcontroller (ESP32/Arduino) interprets gesture signals.
2. **Energy Monitoring Module**  
Uses sensor ACS712  
Measures current flow through appliances.  
Calculates instantaneous power & total energy (Wh).  
Displays values on serial monitor/LCD/Blynk dashboard.
3. **LPG Leakage Detection Module**  
MQ-135 sensor continuously checks gas concentration.  
If gas level crosses threshold, buzzer is activated along with exhaust fan.

### A. Block Diagram of the proposed model system



(fig.1.0)

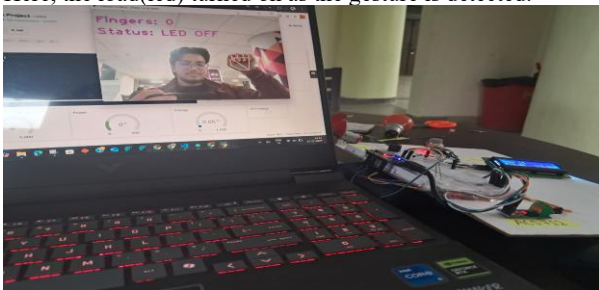
#### WORKING MODEL:

- The system operates by continuously monitoring hand gestures and sensor inputs through the ESP32 microcontroller. When a hand gesture is detected, the microcontroller interprets the pattern of movement and determines the intended control action. Based on the recognized gesture, the system switches the connected appliance (LED) either ON or OFF, providing a touch-free method of control.



(fig.2.0)

Here, the load(led) turned on as the gesture is detected.

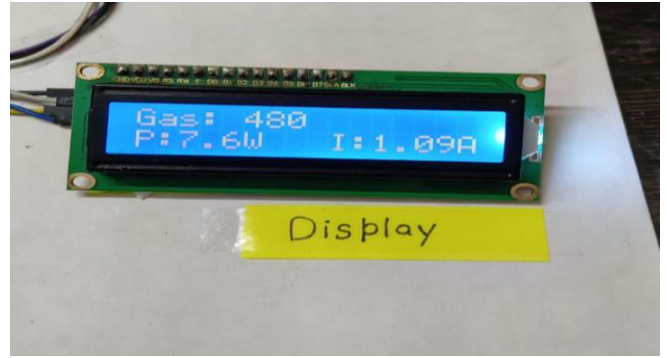


(fig.3.0)

As gesture is detected, the appliance turned off.

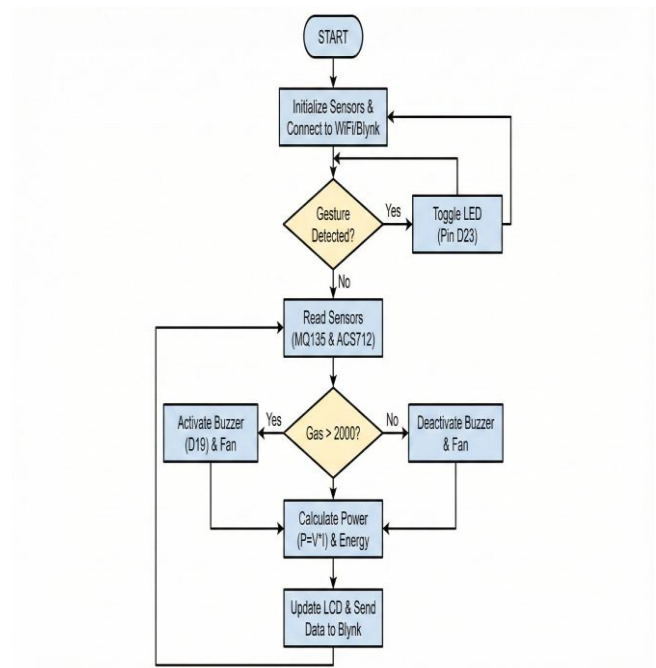
- Simultaneously, the ACS712 current sensor measures the current flowing through the load in real time. Using the measured current and predefined voltage values, the system calculates instantaneous power as well as the total energy consumed over time. These calculated parameters help the user understand power usage and reduce unnecessary consumption.
- To ensure safety, the MQ135 gas sensor continuously monitors the surrounding air for the presence of LPG or smoke. When the gas concentration exceeds the preset threshold value, the system immediately triggers a buzzer to alert the user and automatically switches ON the exhaust fan in order to reduce gas concentration and prevent hazardous conditions.
- All system parameters including gas level, current, power, total energy, and appliance status are displayed live on the LCD and simultaneously updated on the Blynk IoT application via Wi-Fi. The Blynk dashboard presents both real-time readings and graphical visualizations, enabling

users to monitor their home environment remotely with ease.



(fig.4.0)

### B. Flow Chart



(fig. 5.0)

It is a basic simplified flow chart of our model.

#### 1. Start

- The system powers ON and begins execution

#### 2. Initialize Sensors & Connect to Wi-Fi/Blynk

- ESP32 initializes all connected modules:
  - Gesture sensor
  - MQ-135 gas sensor
  - ACS712 current sensor
  - LCD display
- ESP32 connects to Wi-Fi and establishes communication with the Blynk cloud dashboard.

#### 3. Gesture Detection Check

- The system continuously checks whether a hand gesture is detected with the help of AI
  - If gesture is detected :
    - The ESP32 turns on the LED connected to GPIO 23
    - If gesture is not detected:
  - The system proceeds to sensor monitoring.

#### 4. Read Sensors (MQ-135 & ACS712)

- ESP32 reads:
  - Gas concentration value from MQ-135
  - Current value from ACS712

#### 5. Gas Level Decision

- The gas value is compared with a threshold (here, 1200)
  - ▶ If Gas > 1200:
    - Buzzer (GPIO 19) is activated
    - Exhaust fan (DC motor) turned ON
    - This provides an immediate alert and ventilation during gas leakage or smoke.
  - ▶ If Gas ≤ 1200:
    - Buzzer and exhaust fan are turned OFF
    - Normal safe condition continues.

#### 6. Calculate Power and Energy

- Using the current value from ACS712 and known voltage:
  - Power is calculated
  - Energy consumption is calculated over time.

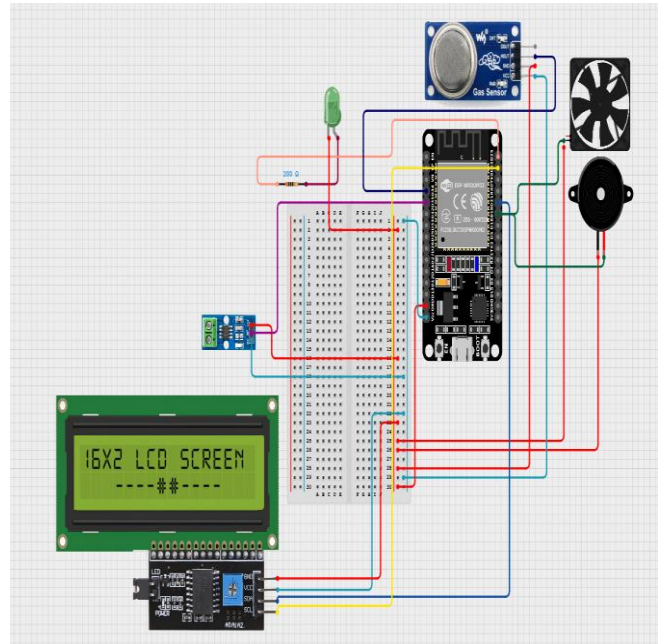
#### 7. Update LCD & Send Data to Blynk

- The following values are displayed in real-time:
  - Gas level
  - Current
  - Power
  - Energy consumption
- The same data is sent wirelessly to the Blynk mobile dashboard for remote monitoring.

#### 8. Loop Continues

- The system continuously repeats the process:
  - Gesture detection
  - Sensor monitoring
  - Safety action
  - Energy calculation
  - Data display & transmission

### C. Circuit Diagram of proposed model



(fig.6.0)

#### A) MQ135 Gas Sensor → ESP32

1. MQ135 VCC → +5 V supply rail
2. MQ135 GND → Common GND rail
3. MQ135 AO (Analog Output) → GPIO34

#### B) ACS712 Current Sensor → ESP32

1. ACS712 VCC → +5 V supply rail
2. ACS712 OUT → GPIO35 of ESP32
3. ACS712 GND → Common GND rail

#### C) 16×2 I<sup>2</sup>C LCD → ESP32

1. LCD VCC → +5 V supply rail
2. LCD GND → Common GND rail
3. LCD SDA → GPIO21 (SDA) of ESP32
4. LCD SCL → GPIO22 (SCL) of ESP32

#### D) LED Load with Series Resistor → ESP32

1. GPIO23 of ESP32 → One end of current-limiting resistor (e.g., 330 Ω)
2. Other end of resistor → Anode (+) of LED / load
3. Cathode (−) of LED / load → Common GND rail

#### E) Buzzer → ESP32

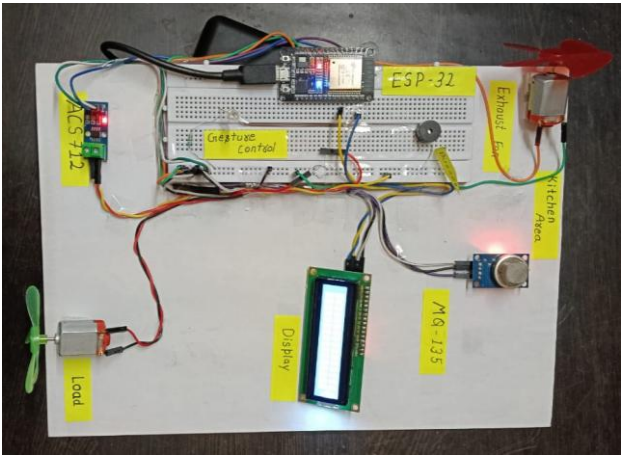
1. Buzzer positive (+) → GPIO19 of ESP32
2. Buzzer negative (−) → Common GND rail

#### F) Exhaust Fan (DC Motor) → ESP32

1. Fan positive (+) → GPIO25 of ESP32
2. Fan negative (−) → Common GND rail



D. Layout of the proposed model



(fig. 7.0)

III. RESULTS

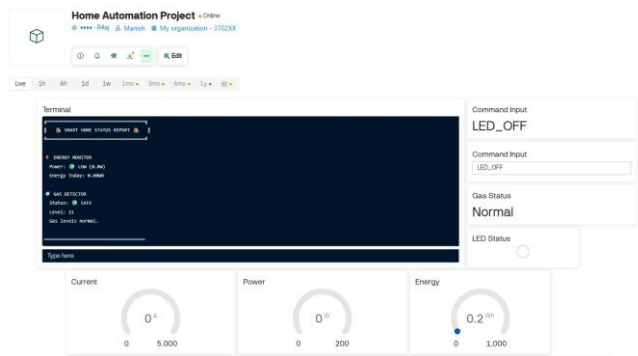
Experimental Results:

| 435 | Current(A) | Power(W) | Cumulative Energy (Wh) | Gas levels |
|-----|------------|----------|------------------------|------------|
| t1  | 1.27       | 8.91     | 1.12                   | 28         |
| t2  | 1.22       | 8.52     | 1.14                   | 413        |
| t3  | 1.31       | 9.16     | 1.16                   | 503        |
| t4  | 1.21       | 8.49     | 1.18                   | 488        |
| t5  | 1.43       | 9.98     | 1.20                   | 435        |

(table 1.0)

From the recorded observations, it is evident that the load current varies between 1.21 A and 1.43 A, resulting in corresponding power variations from 8.49 W to 9.98 W. These variations indicate dynamic load behavior, which is accurately captured by the ACS712 current sensor and processed by the ESP32 in real time. The cumulative energy values show a continuous increase with time, confirming correct energy integration and accumulation by the system. This validates the reliability of the power and energy calculation algorithm implemented in the controller. The gas level readings exhibit noticeable variation across different time instances, demonstrating the sensitivity of the MQ-135 sensor to changes in air quality. Higher gas concentration values indicate the presence of pollutants, justifying the need for immediate alert and ventilation mechanisms. Overall, the tabulated data and corresponding graphs verify that the proposed system effectively monitors current, power, energy consumption, and gas concentration simultaneously, making it suitable for real-time safety and energy management applications.

Graphical Results:



(fig.8.0 Blynk Dashboard)

This is the blynk dashboard which will show instantaneous current value, power value and total energy consumed. It will also give alert when lpq leakage detected and show the appliance status.



(fig.9.0 Gas level v/s Time graph)

The gas level versus time graph shows the real-time variation in gas concentration detected by the MQ-135 sensor. Sudden rises in the graph indicate the presence of harmful gases, validating the sensor's ability to quickly detect leakage conditions and trigger safety mechanisms such as the buzzer and exhaust fan.



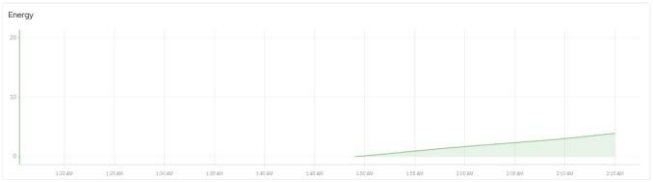
(fig.10.0 Instantaneous Current v/s Time graph)

The current versus time graph represents the instantaneous load current measured using the ACS712 sensor. Variations in the graph correspond to changes in the connected load, demonstrating accurate real-time current monitoring by the system.



(fig.11.0 Power v/s Time graph)

The power versus time graph, derived from the measured current values, illustrates the dynamic power consumption of the load. This graph helps in identifying peak power usage periods and understanding the operational behavior of the connected appliances



(fig.12.0 Total Energy graph v/s Time graph)

The total energy versus time graph shows a continuously increasing trend, confirming correct energy accumulation over time. This graph is useful for analyzing long-term energy consumption and improving energy management and efficiency.

Comparison table

| Features          | Existing Home Automation Systems                 | Proposed Project (Your System)                                 |
|-------------------|--------------------------------------------------|----------------------------------------------------------------|
| Appliance Control | Mostly via manual switches / remote / mobile app | AI enhanced Hand-gesture-based contactless control             |
| Energy Monitoring | Rarely included / external power meter needed    | Built-in real-time monitoring of current, power & total energy |
| Gas / Smoke       | Some systems have                                | Gas detection +                                                |

|                     |                                             |                                                                       |
|---------------------|---------------------------------------------|-----------------------------------------------------------------------|
| Detection           | gas alarm only                              | automatic exhaust fan + buzzer alert                                  |
| IoT Connectivity    | Limited / only device switching             | Blynk IoT app with live values and graphs                             |
| Graphical Analytics | Generally not available                     | Graphs of energy, current, power, and gas level over time             |
| Safety Response     | Alert only (no automatic action)            | Automatic exhaust activation during leakage                           |
| LCD Display         | Optional in few systems                     | Built-in LCD for onsite live display                                  |
| User Accessibility  | Requires remote / app / switch              | Fully touch-free operation for elderly & physically challenged people |
| Cost                | High for multi-module systems               | Low-cost integrated solution with multiple features                   |
| System Integration  | Features available separately, not combined | Automation + safety + energy monitoring in a single system            |

(table 2.0)

#### IV. FUTURE POSSIBLE UPDATES:

1. Addition of multiple loads for gesture control.
2. Turning off the gas knob when the LPG leak detected.
3. Automatic turning off the knob of LPG when outside door is locked.
4. Auto cut of power in case of short circuit.
5. Integration with IoT platforms and alert mechanisms such as SMS, email, or push notifications can improve emergency response and user awareness.
6. Cloud storage of data.

#### V. CONCLUSION

In this project, an ESP32-based integrated gas and current monitoring system has been successfully designed and implemented. The system effectively monitors gas concentration using the MQ-135 sensor and electrical current using the ACS712 sensor, ensuring real-time detection of unsafe environmental and electrical conditions. The measured parameters are continuously displayed on a 16×2 I<sup>2</sup>C LCD, providing clear and instant feedback to the user.

The proposed system also enhances safety by automatically activating alerts and control mechanisms such as an LED indicator, buzzer, and exhaust fan whenever abnormal conditions are detected. The use of the ESP32 microcontroller enables efficient data acquisition, low power consumption, and reliable system performance while maintaining a compact and cost-effective design.

Overall, the project demonstrates a practical and scalable solution for improving environmental safety and energy awareness in domestic, laboratory, and small industrial applications. The system's simplicity, affordability, and expandability make it suitable for real-world implementation and future integration with IoT-based monitoring platforms.

#### VI. REFERENCES

- [1] "Gesture-Based Home Automation Using Arduino," *Electronics For You*, Jul. 2025.
- [2] "Home Automation Using Hand Gestures," *IRE Journals*, vol. X, no. Y, pp. 1–5.
- [3] "Gesture Based Home Automation," *JournalSTD*, Apr. 2024.
- [4] "Research Paper on Home Automation Using Hand Gesture," *IJIRT*, 2024.
- [5] "ACS712 Current Sensor: Features, How It Works, Arduino Guide," Seeed Studio, 2020.
- [6] "ACS712 Current Sensor Module for Arduino – Bidirectional Hall Effect Current Detection," Electroboat, product note.
- [7] "ACS712 30A Ampere Current Sensor Breakout," Sumeet Instruments, product brief.
- [8] "DIY Projects Using the ACS712 Current Sensor," HQ Online, blog article.
- [9] "MQ135 Air Quality Sensor Datasheet: Working and Applications," Elprocus, 2021
- [10] "MQ135 Air Quality/Gas Detector Sensor Module for Arduino," Robu.in, product description.
- [11] "Gas Leakage Detector Using Arduino – Complete DIY Guide," Circuit Digest, Jun. 2025.
- [12] "Evaluating MQ Sensors in Gas Leakage Detection Systems," *Journal of Software & Advanced Engineering Research*, vol. 11, no. 9, 2024
- [13] "Sensor-Based Gas Leakage Detector System," Sathyabama Institute of Science and Technology, project report, 2021.
- [14] "Blynk: Low-Code IoT Platform," Blynk Inc., official website.
- [15] "IoT Home Automation Using Blynk & Node MCU ESP8266," How2Electronics, project tutorial.
- [16] "Smart Home Automation Using Blynk & ESP32 IoT Project," *Instructables*, 2020.
- [17] "Blynk IoT Enabled Smart Home Automation System," *Journal of Computer Science & Communication Security*, Jun. 2024.
- [18] D. Kakani et al., "IoT-Based Smart Home Automation Using ESP32 and Blynk," SSRN preprint, 2024.

# CODE:

- ESP32 code:

```
// =====
// COMBINED CODE: Gesture + MQ135 + ACS72 + LCD
// (CORRECTED FOR BLYNK AI + LCD + ALL FIXES)
// =====
// --- BLYNK CONFIGURATION ---
#define BLYNK_TEMPLATE_ID "TMPL3PpUyWLwl"
#define BLYNK_TEMPLATE_NAME "Home Automation Project"
#define BLYNK_AUTH_TOKEN "-OEwcoIVmfUPMEJ1O05Mbsvd28ONR4aj"
#define BLYNK_PRINT Serial
// --- LIBRARIES ---
#include <WiFi.h>
#include <BlynkSimpleEsp32.h>
#include <LiquidCrystal_I2C.h> // LCD Library
// --- WIFI CREDENTIALS ---
char wifi_ssid[] = "ARJUN";
char wifi_pass[] = "arjun123";
// --- HARDWARE PINS ---
const int LED_PIN = 23; // Gesture LED
const int MQ135_PIN = 34; // Gas Sensor (Analog)
const int ACS712_PIN = 35; // Current Sensor (Analog)
const int BUZZER_PIN = 19; // Buzzer
const int FAN_PIN = 25; // Exhaust fan control (use MOSFET/relay) - changed from 22 to avoid I2C SCL conflict
// --- OBJECTS & VARIABLES ---
BlynkTimer timer;
LiquidCrystal_I2C lcd(0x27, 16, 2); // LCD Object (Address 0x27)
// Sensor Settings
int gasThreshold = 1200;
int gasClearThreshold = 1000; // Hysteresis: fan/buzzer turn off when gas < this
float sensitivity = 0.185;
float vRef = 3.3; // ESP32 is a 3.3V device
// AI Energy Calculation
float homeVoltage = 7.0; // Voltage for AI power calculation
float totalEnergy = 0;
unsigned long lastTime = 0;
float myZeroPoint = 2.3260; // 3.3V / 2. If using 5V ACS712, change this to 2.5
// =====
// SETUP
// =====
void setup() {
  Serial.begin(115200);
  // -- Initialize Pins --
  pinMode(LED_PIN, OUTPUT);
  pinMode(BUZZER_PIN, OUTPUT);
  pinMode(FAN_PIN, OUTPUT);
  pinMode(MQ135_PIN, INPUT);
  pinMode(ACS712_PIN, INPUT);
```

```

digitalWrite(LED_PIN, LOW);
digitalWrite(BUZZER_PIN, LOW);
digitalWrite(FAN_PIN, LOW);
// -- Initialize LCD --
lcd.init();
lcd.backlight();
lcd.setCursor(0, 0);
lcd.print("System Booting..");
Serial.println("System Booting...");
// -- Connect to Blynk --
Blynk.begin(BLYNK_AUTH_TOKEN, wifi_ssid, wifi_pass);

lcd.clear();
lcd.setCursor(0, 0);
lcd.print("Blynk Connected!");
Serial.println("Blynk Connected!");

// -- Initialize V11 (LED Status) --
Blynk.virtualWrite(V11, 0); // Start with LED OFF
delay(100);

delay(1000);
// -- Start Timer (Run sensors every 1000ms) --
timer.setInterval(1000L, runSensors);
lastTime = millis();
}
// =====
// MAIN LOOP
// =====
void loop() {
  Blynk.run();
  timer.run();
}
// =====
// TASK 1: BLYNK GESTURE + LED STATUS
// =====
BLYNK_WRITE(V9) {
  String command = param.asString();

  if (command == "LED_ON") {
    digitalWrite(LED_PIN, HIGH);
    Blynk.virtualWrite(V11, 1); // Send 1 (ON) to V11
    Serial.println("LED ON - V11 updated");

  } else if (command == "LED_OFF") {
    digitalWrite(LED_PIN, LOW);
    Blynk.virtualWrite(V11, 0); // Send 0 (OFF) to V11
    Serial.println("LED OFF - V11 updated");
  }
}
// =====
// TASK 2: SENSORS & DISPLAY
// =====
void runSensors() {
  // --- A. READ SENSORS ---
  int gasValue = analogRead(MQ135_PIN);
  int adcValue = analogRead(ACS712_PIN);
  // --- B. CALCULATE CURRENT (Amps) ---

```



```

float voltage = (adcValue / 4095.0) * vRef;

// * CHANGED THIS *: Use myZeroPoint. Calibrate this value as needed.
float current = (voltage - myZeroPoint) / sensitivity;

// Noise Filter: Force to 0 if value is tiny
if (abs(current) < 0.25) {
  current = 0.00;
}
// --- C. CALCULATE POWER (W) & ENERGY (Wh) ---
// * CHANGED THIS *: Using homeVoltage for AI
float power = homeVoltage * abs(current);

// * ADDED THIS BACK *: Energy calculation for AI
unsigned long currentTime = millis();
float timeStepInHours = (currentTime - lastTime) / 3600000.0;
totalEnergy += power * timeStepInHours;
lastTime = currentTime;
// --- D. UPDATE BUZZER & BLYNK ALERTS ---
// Gas alarm logic with hysteresis: turn on fan and buzzer when gas > gasThreshold
// and turn them off only when gas falls below gasClearThreshold
if (gasValue > gasThreshold) {
  digitalWrite(BUZZER_PIN, HIGH);
  digitalWrite(FAN_PIN, HIGH);
  Blynk.virtualWrite(V1, "ALERT");
} else if (gasValue < gasClearThreshold) {
  // Clear alarm state when safely below the clear threshold
  digitalWrite(BUZZER_PIN, LOW);
  digitalWrite(FAN_PIN, LOW);
  Blynk.virtualWrite(V1, "Normal");
} // otherwise keep previous state until below clear threshold

// --- E. SEND ALL DATA TO BLYNK (FOR AI) ---
// * THIS BLOCK IS NOW INSIDE THE FUNCTION *
Blynk.virtualWrite(V5, gasValue); // Send raw gas to AI
Blynk.virtualWrite(V6, power); // Send power to AI
Blynk.virtualWrite(V7, totalEnergy); // Send energy to AI
Blynk.virtualWrite(V10, abs(current)); // Send current for dashboard
// --- F. UPDATE LCD ---
// * THIS BLOCK IS NOW INSIDE THE FUNCTION *
lcd.clear();
// Row 0: Gas Status
lcd.setCursor(0, 0);
lcd.print("Gas: ");
lcd.print(gasValue);
if(gasValue > gasThreshold) lcd.print(" !"); // Exclamation if danger
// Row 1: Power Info
lcd.setCursor(0, 1);
lcd.print("P:");
lcd.print(power, 1); // Show 1 decimal place
lcd.print("W");

lcd.setCursor(9, 1); // Move to right side
lcd.print("I:");
lcd.print(abs(current), 2); // Show 2 decimal places
lcd.print("A");

// --- G. PRINT TO SERIAL MONITOR ---

```

```
// * THIS LINE IS NOW INSIDE THE FUNCTION *
Serial.printf("Gas: %d | Power: %.1f W | Current: %.2f A | Energy: %.4f Wh\n",
gasValue, power, abs(current), totalEnergy);

} // <-- * THIS IS THE CORRECT, FINAL BRACE for runSensors *
```

- Python Code for gesture based control:
- ❖ ai\_engine.py

```
# Gas (V5) and Energy (V6, V7) for recommendations.

from typing import Dict

def energy_ai(power: float, energy_today: float) -> str:
    """Generates energy status report.

    Args:
        power: Current power usage in Watts
        energy_today: Total energy consumed today in Wh

    Returns:
        Formatted energy status string with power level and daily total
    """
    status = []

    # Current Power Usage
    if power > 150:
        power_status = " 🔴 HIGH"
    elif 50 < power <= 150:
        power_status = " 🟡 MEDIUM"
    else:
        power_status = " 🟢 LOW"

    status.append(f" Power: {power_status} ({power:.1f}W)")

    # Daily Energy Total
    status.append(f" Energy Today: {energy_today:.2f}Wh")

    return "\n".join(status)

def gas_ai(gas: int) -> str:
    """Provides gas level status report.

    Args:
        gas: Gas sensor reading (ADC value or ppm equivalent)

    Returns:
        Formatted gas status string with danger level and recommendations
    """

    if gas > 1200:
        status = " 🔴 DANGER"
        message = "Gas leak detected! Ventilate immediately."
    elif 600 < gas <= 1200:
        status = " 🟡 WARNING"
        message = "High gas level detected."
    else:
        status = " 🟢 SAFE"
        message = "Gas levels normal."
```

```

    return f" Status: {status}\n Level: {gas}\n {message}"

def ai_master(data: Dict[str, float]) -> str:
    """Master function to generate a formatted system status report.

    Args:
        data: Dictionary containing sensor values with keys:
            - 'power': Current power usage (V6) in Watts
            - 'energy_today': Total daily energy (V7) in Wh
            - 'gas': Gas sensor level (V5) in ADC value

    Returns:
        Formatted system status report string with all sensor summaries
    """

    # Get values from data, providing defaults
    power = data.get('power', 0.0) # From V6
    energy_today = data.get('energy_today', 0.0) # From V7
    gas = data.get('gas', 100) # From V5

    # Call the individual status functions
    energy_str = energy_ai(power, energy_today)
    gas_str = gas_ai(gas)

    # Format the final report string
    final_report = (
        "\n" +
        " | 🏠 SMART HOME STATUS REPORT 🏠 | \n" +
        " |-----| \n\n" +
        " ⚡ ENERGY MONITOR\n" +
        f"{energy_str}\n\n" +
        " 🚒 GAS DETECTOR\n" +
        f"{gas_str}\n\n" +
        "-----" +
    )

    return final_report

```

## ❖ ai\_to\_blynk.py :

```

# Reads V5 (gas), V6 (power), V7 (energy) for AI recommendations.

import requests
import time
import json

# Import the master AI function from our other file
from ai_engine import ai_master

# --- BLYNK CONFIGURATION ---
BLYNK_AUTH_TOKEN = "-OEwcoIVmfUPMEJ1O05Mbsvd28ONR4aj" # Paste your token here
BLYNK_GET_URL = f"https://blynk.cloud/external/api/get?token={BLYNK_AUTH_TOKEN}"
BLYNK_UPDATE_URL = f"https://blynk.cloud/external/api/update?token={BLYNK_AUTH_TOKEN}"

# --- HELPER FUNCTIONS ---
def blynk_get_value(pin):
    """Gets the raw value from a Blynk virtual pin and parses JSON response."""
    try:
        response = requests.get(f"{BLYNK_GET_URL}&pin={pin}")
        if response.status_code == 200:
            raw = response.text.strip()
            # Blynk API returns JSON array: ["value"]

```

```
try:
    parsed = json.loads(raw)
    if isinstance(parsed, list) and len(parsed) > 0:
        return str(parsed[0])
    return str(parsed)
except (json.JSONDecodeError, IndexError):
    # Fallback: return raw string if parsing fails
    return raw
else:
    print(f"Error getting {pin}: Status {response.status_code}")
    return "0"

except requests.exceptions.RequestException as e:
    print(f"Error getting {pin}: {e}")
    return "0"

def blynk_send_report(report_string):
    """Sends the final report string to the V8 Terminal."""
    try:
        payload = {'v8': report_string}
        response = requests.get(BLYNK_UPDATE_URL, params=payload)
        print(f"✓ Report sent to Blynk V8. Response: {response.status_code}")
        return True
    except requests.exceptions.RequestException as e:
        print(f"Error sending to V8: {e}")
        return False

# --- MAIN SYSTEM MONITORING LOOP ---
# System generates and sends SMART HOME STATUS REPORT automatically every 30 seconds
# No trigger required - runs continuously and updates Blynk dashboard

print("\n ┌──────────────────────────────────────────────────────────┐")
print(" │ 🏠 SMART HOME DATA ENGINE 🇮🇹 │ ")
print(" │ (Auto-generating reports...) │ ")
print(" └──────────────────────────────────────────────────────────┘")
print("\n ✅ Reports will be generated automatically every 30 seconds.\n")

report_interval = 10 # Generate report every 30 seconds
last_report_time = 0

while True:
    current_time = time.time()

    # Check if it's time to generate a new report
    if current_time - last_report_time >= report_interval:
        print("\n" + "="*40)
        print(f" 🔄 Generating Report at {time.strftime('%H:%M:%S')}")
        print(" Fetching System Data...")
        print("="*40)

        # 1. Initialize data dictionary
        sensor_data = {}

        # 2. Fetch all sensor data from Blynk
        try:
            sensor_data['gas'] = int(blynk_get_value('v5'))
            sensor_data['power'] = float(blynk_get_value('v6'))
            sensor_data['energy_today'] = float(blynk_get_value('v7'))
        except ValueError as e:
            print(f" ⚠️ Sensor Error: {e}")
            # Continue with defaults defined in ai_engine

        # 3. Generate formatted status report
        report = ai_master(sensor_data)
```

```

# 4. Display report locally
print(report)

# 5. Send the report to Blynk V8
if blynk_send_report(report):
    print("✓ Report successfully sent to Blynk V8")
    last_report_time = current_time
else:
    print("⚠ Failed to send report to Blynk V8")
else:
    # System idle - waiting for next report generation
    remaining = int(report_interval - (current_time - last_report_time))
    print(f".", end="", flush=True)

# Check every 2 seconds
time.sleep(2)

```

#### ❖ gesture\_to\_blynk.py:

```

# This script controls an LED connected to an ESP32 via Blynk Cloud.
# - Shows webcam feed with hand tracking.
# - Counts fingers:
#   - 5 Fingers (Open Palm) -> Sends "LED_ON" to Blynk V9
#   - 0 or 1 Finger (Fist) -> Sends "LED_OFF" to Blynk V9
# - Includes smoothing to prevent spamming the Blynk server.

import cv2
import mediapipe as mp
import requests
import time

# --- BLYNK CONFIGURATION ---
BLYNK_AUTH_TOKEN = "-OEwcoIVmfUPMEJ1O05Mbsvd28ONR4aj" # Paste your token here
BLYNK_VIRTUAL_PIN = "v9"
BLYNK_API_URL = f"https://blynk.cloud/external/api/update?token={BLYNK_AUTH_TOKEN}"

# --- MEDIAPIPE HAND TRACKING SETUP ---
mp_hands = mp.solutions.hands
hands = mp_hands.Hands(min_detection_confidence=0.7, min_tracking_confidence=0.5)
mp_draw = mp.solutions.drawing_utils

# Finger tip landmark IDs
tip_ids = [4, 8, 12, 16, 20]

# --- BLYNK COMMUNICATION ---
def send_to_blynk(command):
    """Sends the given command string to Blynk V9 with error handling."""
    try:
        url = f"{BLYNK_API_URL}&{BLYNK_VIRTUAL_PIN}={command}"
        response = requests.get(url, timeout=5)
        if response.status_code == 200:
            print(f"✓ Sent '{command}' to Blynk.")
            return True
        else:
            print(f"⚠ Blynk response {response.status_code} for '{command}'")
            return False
    except requests.exceptions.Timeout:
        print(f"⚠ Timeout sending '{command}' to Blynk")
        return False
    except requests.exceptions.RequestException as e:
        print(f"⚠ Error sending to Blynk: {e}")

```

```

        return False

# --- MAIN GESTURE DETECTION LOOP ---
cap = cv2.VideoCapture(0) # 0 is the default webcam
last_command = ""        # For smoothing
last_sent_time = 0

while cap.isOpened():
    success, img = cap.read()
    if not success:
        print("Failed to grab frame from webcam.")
        break

    # Flip the image horizontally for a natural, mirror-like view
    img = cv2.flip(img, 1)

    # Convert from BGR (OpenCV) to RGB (MediaPipe)
    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    # Process the image and find hands
    results = hands.process(img_rgb)

    finger_count = 0
    status_text = "No Hand"
    command = ""

    if results.multi_hand_landmarks:
        # Get landmarks for the first hand detected
        hand_lms = results.multi_hand_landmarks[0]

        # Draw hand landmarks on the image
        mp_draw.draw_landmarks(img, hand_lms, mp_hands.HAND_CONNECTIONS)

        # --- FINGER COUNTING LOGIC ---
        fingers = []
        landmarks = hand_lms.landmark

        # 1. Thumb
        # Check if thumb tip is to the left of the thumb's next joint
        # (This works because the image is flipped)
        if landmarks[tip_ids[0]].x < landmarks[tip_ids[0] - 1].x:
            fingers.append(1)
        else:
            fingers.append(0)

        # 2. Other 4 Fingers
        # Check if finger tip's y-coordinate is above the joint 2 landmarks down
        for id in range(1, 5):
            if landmarks[tip_ids[id]].y < landmarks[tip_ids[id] - 2].y:
                fingers.append(1)
            else:
                fingers.append(0)

        finger_count = fingers.count(1)

        # --- COMMAND LOGIC ---
        if finger_count == 5:
            status_text = "LED ON"
            command = "LED_ON"
        elif finger_count <= 1:
            status_text = "LED OFF"
            command = "LED_OFF"
        else:
            status_text = "..."
            command = "" # No valid command

```



```

# --- SMOOTHING & SENDING TO BLYNK ---
# Only send a new command if it's different from the last one
# AND at least 1 second has passed (prevents flickering)
current_time = time.time()
if command and command != last_command and (current_time - last_sent_time > 1):
    if send_to_blynk(command):
        last_command = command
        last_sent_time = current_time
elif not command:
    last_command = "" # Reset if no valid gesture

# --- DISPLAY INFO ON SCREEN ---
cv2.putText(img, f"Fingers: {finger_count}", (10, 70), cv2.FONT_HERSHEY_PLAIN, 3, (255, 0, 255), 3)
cv2.putText(img, f"Status: {status_text}", (10, 130), cv2.FONT_HERSHEY_PLAIN, 3, (255, 0, 255), 3)

cv2.imshow("Gesture Control - (Press 'q' to quit)", img)

key = cv2.waitKey(5) & 0xFF
if key == ord('q') or key == 27: # 'q' or ESC to quit
    break

# Cleanup
cap.release()
cv2.destroyAllWindows()
hands.close()

```